

گزارش تمرین سوم درس رایانش سیستم‌های توزیع شده

سروش اصفهانیان - 810101376 نوید هاشمی - 810101549

بخش اول: سوالات تئوری

چرا داده‌ها در سیستم‌های توزیع شده replicate می‌شوند؟

در سیستم‌های توزیع شده داده‌ها معمولاً روی چندین گره یا سرور کپی می‌شوند تا قابلیت دسترسی، تحمل خطا و کارایی سیستم افزایش یابد. در این حالت اگر یکی از سرورها از کار بیفتد، نسخه‌های دیگر همچنان می‌توانند به درخواست کاربران پاسخ دهند و از دست رفتن داده‌ها جلوگیری شود. علاوه بر این، وجود چندین نسخه از داده باعث می‌شود بار درخواست‌ها بین سرورها توزیع شود و کاربران بتوانند از نزدیک‌ترین یا کم‌لودترین سرور سرویس دریافت کنند که در نهایت زمان پاسخ سیستم را کاهش می‌دهد.

تفاوت replication برای افزایش کارایی و replication برای تحمل خطا چیست؟

زمانی که replication با هدف افزایش کارایی انجام می‌شود، هدف اصلی توزیع لود کاری بین چندین سرور و کاهش زمان پاسخ است؛ به این ترتیب کاربران می‌توانند درخواست‌های خود را از نزدیک‌ترین یا کم‌لودترین replica دریافت کنند. اما در replication برای تحمل خطا، هدف اصلی حفظ سرویس و جلوگیری از از دست رفتن داده‌ها در صورت خرابی یک یا چند سرور است. در این حالت، حتی اگر بخشی از سیستم هم از کار بیفتد، replicaهای دیگر همچنان قادر به ارائه سرویس خواهند بود و سیستم به کار خود ادامه می‌دهد.

مشکل اصلی replication از نظر consistency چیست؟

مهم‌ترین مشکل replication حفظ سازگاری یا همان consistency بین replicaها است. زمانی که داده‌ای در یکی از replicaها تغییر می‌کند، این تغییر باید به سایر replicaها نیز منتقل شود. از آنجا که انتقال داده در شبکه زمان‌بر است و ممکن است با تأخیر یا خطا همراه باشد، در یک بازه زمانی ممکن است replicaهای مختلف مقادیر متفاوتی از یک داده را نگهداری کنند. در نتیجه، کاربران ممکن است بسته به اینکه از کدام replica داده را بخوانند، پاسخ‌های متفاوتی دریافت کنند که به این وضعیت ناسازگاری داده گفته می‌شود.

تفاوت eventual consistency و strong consistency چیست؟

در مدل Strong Consistency پس از انجام یک عملیات نوشتن، همه عملیات خواندن بعدی آخرین مقدار نوشته شده را مشاهده می‌کنند و کاربر هیچ‌گاه داده قدیمی دریافت نمی‌کند. این مدل سازگاری بالایی دارد اما معمولاً باعث افزایش تأخیر و کاهش دسترسی‌پذیری سیستم می‌شود. در مقابل، در Eventual Consistency تغییرات ابتدا روی بخشی از replicaها اعمال می‌شوند و سپس به تدریج به سایر replicaها منتقل می‌گردند. بنابراین ممکن است برای مدت کوتاهی برخی کاربران داده قدیمی مشاهده کنند، اما در نهایت همه replicaها به یک وضعیت یکسان همگرا خواهند شد. این مدل دسترسی‌پذیری و کارایی بیشتری دارد، اما احتمال مشاهده داده‌های قدیمی در آن وجود دارد.

مفهوم read-your-writes consistency چیست؟

Read-your-writes consistency یکی از مدل‌های سازگاری سمت کاربر است که تضمین می‌کند اگر یک کاربر داده‌ای را تغییر دهد، تمام خواندن‌های بعدی همان کاربر حداقل همان تغییر را مشاهده خواهند کرد. به عبارت دیگر، کاربر هرگز پس از انجام یک عملیات نوشتن، نسخه قدیمی‌تر داده خود را نخواهد دید؛ حتی اگر سایر کاربران هنوز تغییر جدید را مشاهده نکنند. این ویژگی باعث می‌شود تجربه کاربر طبیعی‌تر و بهتر و قابل پیش‌بینی‌تر باشد.

مفهوم monotonic reads چیست؟

Monotonic Reads تضمین می‌کند که اگر یک کاربر در یک بار خواندن نسخه‌ای از داده را مشاهده کند، در خواندن‌های بعدی همان کاربر هرگز نسخه قدیمی‌تری از آن داده را نخواهد دید. برای مثال اگر کاربر ابتدا نسخه ۵ یک داده را بخواند، در آینده ممکن است نسخه ۶ یا جدیدتر را ببیند، اما نباید دوباره نسخه ۴ را مشاهده کند. این مدل از بازگشت به داده‌های قدیمی جلوگیری می‌کند و باعث می‌شود وضعیت داده‌ها برای کاربر به صورت رو به جلو پیش برود.

چرا در سیستم‌های توزیع‌شده نمی‌توان همزمان همیشه بیشترین consistency و availability و partition tolerance را به صورت کامل داشت؟

طبق قضیه CAP، در یک سیستم توزیع‌شده نمی‌توان به‌طور همزمان و در همه شرایط هر سه ویژگی، Consistency، Availability و Partition Tolerance را به‌صورت کامل تضمین کرد. هنگامی که بین بخش‌های مختلف شبکه یک Partition یا قطعی ارتباط رخ می‌دهد، سیستم مجبور است بین دو گزینه انتخاب کند: یا برای حفظ Consistency از پاسخ دادن به برخی درخواست‌ها خودداری کند و Availability را کاهش دهد، یا برای حفظ Availability به درخواست‌ها پاسخ دهد، حتی اگر برخی پاسخ‌ها شامل داده‌های قدیمی باشند و Consistency کاهش یابد. به همین دلیل، سیستم‌های توزیع‌شده معمولاً بسته به نیاز خود بین این ویژگی‌ها نوعی Trade-off برقرار می‌کنند.

بخش دوم: پیاده‌سازی Replicated Key-Value Store

در این بخش یک سیستم ساده‌ی Replicated Key-Value Store با استفاده از زبان Go و پروتکل HTTP پیاده‌سازی شده است. سیستم شامل سه Replica مستقل است که هر کدام به‌صورت یک فرآیند جداگانه اجرا می‌شوند و داده‌های خود را به‌صورت محلی نگهداری می‌کنند. هر Replica دارای فایل کانفیگ مخصوص به خود است که شامل id، پورت و آدرس Replicaهای دیگر می‌شود. ارتباط بین Replicaها از طریق درخواست‌های HTTP برقرار می‌شود و هیچ متغیر یا حافظه‌ی مشترکی بین آنها نیز وجود ندارد.

هر Replica سه Endpoint اصلی ارائه می‌کند. Endpoint اول با آدرس /put برای ذخیره یا به‌روزرسانی مقدار یک کلید استفاده می‌شود. Endpoint دوم با آدرس /get برای خواندن مقدار یک کلید است و Endpoint سوم با آدرس /sync برای تبادل داده و انجام عملیات Replication بین Replicaها مورد استفاده قرار می‌گیرد. در هنگام فرخوانی هر کدام از endpointها، تابع هندلر مربوط به هر کدام صدا زده می‌شود:

```
func main() {
    configPath := flag.String("config", "", "Path to config file (e.g., configs/replica1.json)")
    flag.StringVar(&consistency, "consistency", "eventual", "Consistency model: 'eventual' or 'strong'")
    flag.IntVar(&delay, "delay", 0, "Artificial delay for replication in ms")
    flag.Parse()

    if *configPath == "" {
        log.Fatal("Config file path is required. Use -config flag.")
    }

    // Read from config file
    file, err := os.Open(*configPath)
    if err != nil {
        log.Fatalf("Failed to open config: %v", err)
    }
    defer file.Close()
    json.NewDecoder(file).Decode(&config)

    http.HandleFunc("/put", handlePut)
    http.HandleFunc("/get", handleGet)
    http.HandleFunc("/sync", handleSync)

    log.Printf("Starting %s on port %s | Consistency: %s | Delay: %dms | Peers: %v\n", config.ID, config.Port, consistency, delay, config.Peers)
    http.ListenAndServe(":"+config.Port, nil)
}
```

برای نگهداری داده‌ها از ساختار Record استفاده شده است که شامل چهار فیلد key، value، version و updated_by است. داده‌ها در یک Map محلی ذخیره می‌شوند و برای جلوگیری از مشکلات همزمانی از RWMutex استفاده شده است. هر Replica فقط به داده‌های محلی خود دسترسی دارد و تغییرات از طریق شبکه به سایر Replicaها ارسال می‌شود.

```
type Record struct {
    Key      string `json:"key"`
    Value    string `json:"value"`
    Version  int    `json:"version"`
    UpdatedBy string `json:"updated_by"`
}

var (
    store = make(map[string]Record)
    rwMutex sync.RWMutex

    config Config
    consistency string
    delay int
)
```

پیاده‌سازی مدل Eventual Consistency

در مدل Eventual Consistency، پس از دریافت درخواست Replica، PUT دریافت‌کننده ابتدا مقدار جدید را در حافظه محلی خود ذخیره می‌کند و سپس بدون منتظر ماندن برای پاسخ سایر Replicaها، عملیات Replication را به صورت غیرهمزمان آغاز می‌کند. بنابراین ممکن است برای مدت کوتاهی Replicaهای مختلف مقادیر متفاوتی داشته باشند، اما در نهایت همه‌ی آن‌ها به یک وضعیت مشترک همگرا می‌شوند.

برای پیاده‌سازی این مدل از Goroutine استفاده شده است تا عملیات ارسال پیام‌های Replication به سایر Replicaها به صورت Asynchronous انجام شود. همچنین امکان اضافه کردن تأخیر مصنوعی شبکه نیز برای بررسی رفتار سیستم در شرایط مختلف فراهم شده است.

```
} else {
    for _, peer := range config.Peers {
        if peer == "" { continue }
        go func(p string) {
            time.Sleep(time.Duration(delay) * time.Millisecond)
            http.Post(p+"/sync", "application/json", bytes.NewBuffer(jsonData))
        }(peer)
    }

    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(newRecord)
}
```

پیاده‌سازی مدل Strong Consistency

برای پیاده‌سازی Strong Consistency از یک مکانیزم ساده مبتنی بر Quorum استفاده شده است. در سیستم حاضر سه Replica وجود دارد؛ بنابراین حداقل دو Replica باید عملیات PUT را تأیید کنند تا عملیات موفق در نظر گرفته شود.

در این روش، Replica دریافت‌کننده درخواست PUT ابتدا پیام Sync را برای سایر Replicaها ارسال می‌کند و منتظر پاسخ آن‌ها می‌ماند. اگر حداقل دو Replica شامل Replica اصلی عملیات را با موفقیت ثبت کنند، درخواست موفقیت‌آمیز خواهد بود. در غیر این صورت، درخواست با خطا مواجه می‌شود و پیام Failed to reach quorum به Client بازگردانده می‌شود.

برای پیاده‌سازی این مکانیزم از WaitGroup و Mutex استفاده شده است تا تعداد پاسخ‌های موفق به‌صورت Thread-safe شمارش شود.

```
func handlePut(w http.ResponseWriter, r *http.Request) {  
    if consistency == "strong" {  
        successCount := 1  
        var wg sync.WaitGroup  
        var mu sync.Mutex  
  
        for _, peer := range config.Peers {  
            if peer == "" { continue }  
            wg.Add(1)  
            go func(p string) {  
                defer wg.Done()  
                time.Sleep(time.Duration(delay) * time.Millisecond)  
                client := http.Client{Timeout: 5 * time.Second}  
                resp, err := client.Post(p+"/sync", "application/json", bytes.NewBuffer(jsonData))  
                if err == nil && resp.StatusCode == http.StatusOK {  
                    mu.Lock()  
                    successCount++  
                    mu.Unlock()  
                }  
            }(peer)  
        }  
        wg.Wait()  
  
        if successCount < 2 {  
            http.Error(w, "Failed to reach quorum", http.StatusInternalServerError)  
            return  
        }  
    } else {
```

نحوه اجرای Replica ها

هر Replica با فایل پیکربندی مخصوص خود اجرا می‌شود. برای مثال Replica اول با دستور زیر اجرا می‌شود:

go run replica/main.go -config=configs/replica1.json -consistency=eventual

به همین ترتیب Replica دوم و Replica سوم نیز با فایل‌های پیکربندی مربوط به خود اجرا می‌شوند.

نحوه اجرای Client

برای ارسال درخواست‌ها و اندازه‌گیری متریک‌ها یک برنامه Client جداگانه پیاده‌سازی شده است که با استفاده از دستور go run client/main.go اجرا می‌شود. این Client قادر است عملیات PUT و GET را روی Replica های مختلف انجام دهد و متریک‌هایی مانند زمان پاسخ PUT، زمان پاسخ GET، تعداد Stale Read ها و زمان همگرایی Replica ها را اندازه‌گیری کند.

```
func main() {  
    key := "test_key"  
    expectedValue := "latency_test_1"  
  
    // Measure PUT Latency  
    startPut := time.Now()  
    putData := []byte(fmt.Sprintf(`{"key": "%s", "value": "%s"}`, key, expectedValue))  
    resp, err := http.Post("http://localhost:8081/put", "application/json", bytes.NewBuffer(putData))  
    putLatency := time.Since(startPut)  
  
    if err != nil || resp.StatusCode != http.StatusOK {  
        fmt.Printf("PUT failed: %v\n", err)  
        return  
    }  
    fmt.Printf("PUT Latency: %v\n", putLatency)  
  
    // Measure GET Latency & Stale Reads & Convergence Time  
    staleReads := 0  
    startGet := time.Now()
```

```

for {
    // Read from Replica 2 immediately after writing to Replica 1
    resp, err := http.Get("http://localhost:8082/get?key=" + key)
    if err != nil {
        time.Sleep(50 * time.Millisecond)
        continue
    }

    var rec Record
    json.NewDecoder(resp.Body).Decode(&rec)
    resp.Body.Close()

    if rec.Value != expectedValue {
        staleReads++
        time.Sleep(50 * time.Millisecond) // Poll every 50ms
    } else {
        convergenceTime := time.Since(startGet)
        fmt.Printf("GET Latency (last successful): ~%v\n", time.Since(startGet)/time.Duration(staleReads+1))
        fmt.Printf("Stale Reads Count: %d\n", staleReads)
        fmt.Printf("Convergence Time: %v\n", convergenceTime)
        break
    }
}

```

بخش سوم: نسخه‌گذاری و حل تعارض

برای هر داده ذخیره‌شده در سیستم، اطلاعات نسخه نیز نگهداری می‌شود. ساختار داده مورد استفاده شامل چهار فیلد، key، value، version و updated_by است. فیلد version شماره نسخه فعلی داده را مشخص می‌کند و فیلد updated_by شناسه Replica ای را که آخرین تغییر را اعمال کرده است، نگهداری می‌کند.

هر بار که عملیات PUT روی یک کلید انجام می‌شود، سیستم ابتدا بررسی می‌کند که آیا آن کلید قبلاً وجود داشته است یا خیر. اگر کلید جدید باشد، نسخه آن برابر ۱ قرار می‌گیرد و اگر قبلاً وجود داشته باشد، شماره نسخه یک واحد افزایش پیدا می‌کند. به این ترتیب تمامی تغییرات روی هر کلید قابل ردیابی خواهند بود.

```

func handlePut(w http.ResponseWriter, r *http.Request) {
    var reqData map[string]string
    json.NewDecoder(r.Body).Decode(&reqData)
    key, value := reqData["key"], reqData["value"]

    rwMutex.Lock()
    existingRecord, exists := store[key]
    newVersion := 1
    if exists {
        newVersion = existingRecord.Version + 1
    }

    newRecord := Record{Key: key, Value: value, Version: newVersion, UpdatedBy: config.ID}
    store[key] = newRecord
    rwMutex.Unlock()
}

```

هنگامی که یک Replica پیامی از طریق اندپوینت /sync دریافت می‌کند، ابتدا نسخه داده دریافتی را با نسخه موجود در حافظه محلی مقایسه می‌کند. اگر نسخه داده دریافتی بزرگ‌تر باشد، مقدار جدید جایگزین مقدار قبلی می‌شود. در صورتی که

نسخه داده دریافتی کوچکتر باشد، پیام نادیده گرفته می‌شود و داده موجود تغییر نمی‌کند. این مکانیزم مانع از آن می‌شود که داده‌های قدیمی‌تر، داده‌های جدیدتر را بازنویسی کنند.

```
func handleSync(w http.ResponseWriter, r *http.Request) {
    var incoming Record
    json.NewDecoder(r.Body).Decode(&incoming)

    rwMutex.Lock()
    defer rwMutex.Unlock()

    current, exists := store[incoming.Key]
    if !exists || incoming.Version > current.Version {
        store[incoming.Key] = incoming
    } else if incoming.Version == current.Version && incoming.UpdatedBy > current.UpdatedBy {
        store[incoming.Key] = incoming
    }
    w.WriteHeader(http.StatusOK)
}
```

همان طور که در تصویر بالا دیده می‌شود، برای حل تعارض همزمان، سیاست اینکه Replica با شناسه بزرگ‌تر برنده بشود، انتخاب شده است. در این روش اگر دو Replica مقدارهای متفاوتی را با شماره نسخه یکسان برای یک کلید تولید کنند، مقدار مربوط به Replica‌ای که شناسه بزرگ‌تری دارد در سیستم باقی می‌ماند.

به عنوان مثال، اگر Replica1 مقدار A و Replica2 مقدار B را با نسخه یکسان برای کلید conflict ثبت کنند، از آنجا که شناسه replica2 از replica1 بزرگ‌تر است، مقدار B به عنوان مقدار نهایی انتخاب می‌شود و سایر Replicaها نیز در نهایت همین مقدار را ذخیره خواهند کرد این سیاست ساده، قطعی و قابل پیش‌بینی است و از باقی ماندن سیستم در وضعیت ناسازگار جلوگیری می‌کند.

بخش چهارم: آزمایش و تحلیل

در این بخش رفتار سیستم پیاده‌سازی شده تحت شرایط مختلف مورد بررسی قرار گرفته است. هدف از این آزمایش‌ها مشاهده تفاوت مدل‌های Eventual Consistency و Strong Consistency و بررسی رفتار سیستم هنگام خرابی Replicaها و مدیریت تعارض‌های همزمان و بررسی تأثیر تأخیر شبکه بر عملکرد سیستم است.

سناریوی اول: مشاهده ناسازگاری موقت

در این آزمایش ابتدا مقدار $x=100$ روی Replica1 نوشته شد. بلافاصله پس از آن، مقدار کلید x از Replica2 خوانده شد. در این لحظه Replica2 هنوز پیام Replication را دریافت نکرده بود و بنابراین داده موردنظر یافت نشد. سپس چند ثانیه صبر شد تا فرآیند Replication کامل شود و مجدداً مقدار x از Replica2 خوانده شد. این بار Replica2 مقدار جدید را نمایش داد. روند گفته شده در زیر قابل مشاهده است:

```
--- PUT x=100 in Replica 1 ---
{"key": "x", "value": "100", "version": 1, "updated_by": "replica1"}

--- Immediate GET from Replica 2 (Expecting old/no data) ---
Key not found

--- Waiting 3 seconds... ---

--- GET from Replica 2 after delay (Expecting updated data) ---
{"key": "x", "value": "100", "version": 1, "updated_by": "replica1"}
```

نتایج این آزمایش نشان می‌دهد که در مدل Eventual Consistency ممکن است برای مدت کوتاهی Replica‌های مختلف وضعیت یکسانی نداشته باشند و کاربر داده قدیمی یا حتی عدم وجود داده را مشاهده کند. با این حال، پس از گذشت زمان و تکمیل فرآیند Replication، تمامی Replica‌ها به یک وضعیت مشترک رسیده اند که این رفتار، دقیقاً مطابق تعریف Eventual Consistency است.

سناریوی دوم: خرابی یک Replica

در این آزمایش ابتدا هر سه Replica اجرا شدند و سپس Replica2 متوقف شد. پس از آن یک عملیات PUT روی Replica1 انجام شد و رفتار سیستم در دو مدل Eventual Consistency و Strong Consistency بررسی شد.

در مدل Eventual Consistency عملیات PUT با موفقیت انجام شد، زیرا Replica1 بدون نیاز به دریافت تأیید از سایر Replica‌ها مقدار جدید را ذخیره کرد و تلاش نمود تغییرات را برای Replica‌های در دسترس ارسال کند.

در مدل Strong Consistency نیز به دلیل در دسترس بودن Replica1 و Replica3، اکثریت Replica‌ها همچنان وجود داشتند و عملیات PUT موفق بود. در این پیاده‌سازی اکثریت برابر با دو Replica در نظر گرفته شده است، و بنابراین تا زمانی که حداقل دو Replica در دسترس باشند، سیستم می‌تواند درخواست‌های نوشتن را بپذیرد. نتیجه اجرای این سناریو به صورت زیر می‌باشد:

```
--- Testing Eventual Consistency with Replica 2 Offline ---
HTTP/1.1 200 OK
Date: Mon, 15 Jun 2026 16:05:12 GMT
Content-Length: 79
Content-Type: text/plain; charset=utf-8

{"key":"test2","value":"eventual_success","version":1,"updated_by":"replica1"}

--- Testing Strong Consistency with Replica 2 Offline ---
HTTP/1.1 200 OK
Date: Mon, 15 Jun 2026 16:07:56 GMT
Content-Length: 74
Content-Type: text/plain; charset=utf-8

{"key":"test2","value":"strong_fail","version":1,"updated_by":"replica1"}
```

این آزمایش نشان می‌دهد که مدل Eventual Consistency دسترس‌پذیری بالاتری دارد، زیرا حتی بدون دریافت تأیید سایر Replica‌ها نیز عملیات نوشتن را پذیرفته است. در مقابل، Strong Consistency برای حفظ سازگاری نیازمند وجود اکثریت Replica‌ها است و در صورت از دست رفتن اکثریت، عملیات نوشتن با شکست مواجه می‌شود.

سناریوی سوم: تعارض همزمان

در این سناریو تقریباً به صورت همزمان دو درخواست PUT روی دو Replica مختلف برای یک کلید یکسان ارسال شد. Replica1 مقدار A و Replica2 مقدار B را برای کلید conflict ذخیره کردند. پس از تبادل پیام‌های Replication، وضعیت نهایی همه Replica‌ها بررسی شد.

نتایج (صفحه بعد) نشان می‌دهد که مقدار نهایی ذخیره شده در هر سه Replica برابر با مقدار B است و فیلد updated_by برابر replica2 می‌باشد. علت این رفتار، سیاست حل تعارض انتخاب شده در سیستم است که در بخش‌های قبل توضیح داده شد. در این پیاده‌سازی اگر دو مقدار دارای شماره نسخه یکسان باشند، Replica با شناسه بزرگ‌تر برنده خواهد شد. از آنجا که

replica2 از replica1 بزرگتر است، مقدار ثبت شده توسط replica2 در کل سیستم منتشر شده و در نهایت تمامی Replica ها به همین مقدار همگرا شده اند.

این سیاست باعث می شود سیستم بتواند تعارض های همزمان را به صورت قطعی و قابل پیش بینی مدیریت کند و از باقی ماندن Replica ها در وضعیت ناسازگار جلوگیری شود. نتیجه اجرای این سناریو به صورت زیر است:

```
--- Sending Concurrent PUT Requests ---

--- Final State of Replicas (Resolution policy: Greater ID wins) ---

Replica 1:
{"key": "conflict", "value": "B", "version": 1, "updated_by": "replica2"}

Replica 2:
{"key": "conflict", "value": "B", "version": 1, "updated_by": "replica2"}

Replica 3:
{"key": "conflict", "value": "B", "version": 1, "updated_by": "replica2"}
```

سناریوی چهارم: تأثیر تأخیر شبکه

در این آزمایش برای ارسال پیام های Replication تأخیر مصنوعی در نظر گرفته شد و رفتار سیستم در سه حالت مختلف بررسی گردید. در حالت اول، تأخیر شبکه برابر صفر میلی ثانیه بود و Replica ها تقریباً بلافاصله به وضعیت یکسان می رسیدند.

در حالت دوم، تأخیر شبکه برابر ۵۰۰ میلی ثانیه قرار داده شد. در این حالت زمان همگرایی Replica ها حدود ۶۰۳ میلی ثانیه اندازه گیری شد و تعداد ۷ بار Stale Read مشاهده گردید. با وجود این، زمان پاسخ PUT بسیار پایین باقی ماند، زیرا در مدل Eventual Consistency عملیات نوشتن منتظر تکمیل Replication نمی ماند.

```
-delay=500ms

PUT Latency: 7.988044ms
GET Latency (last successful): ~75.350045ms
Stale Reads Count: 7
Convergence Time: 602.800316ms
```

در حالت سوم، تأخیر شبکه برابر ۲۰۰۰ میلی ثانیه در نظر گرفته شد. نتایج نشان داد که زمان یکسان شدن وضعیت Replica ها به حدود ۲ ثانیه افزایش یافته و تعداد Stale Read ها به ۲۸ مورد رسیده است. این موضوع نشان می دهد که افزایش تأخیر شبکه مستقیماً بر زمان همگرایی Replica ها و احتمال مشاهده داده های قدیمی تأثیر می گذارد.

```
-delay=2000ms

PUT Latency: 1.487069ms
GET Latency (last successful): ~70.959902ms
Stale Reads Count: 28
Convergence Time: 2.057837098s
```

در ادامه همین آزمایش، مدل Strong Consistency با تأخیر ۵۰۰ میلی ثانیه بررسی شد که نتایج نشان داد که زمان پاسخ PUT به حدود ۷۰۲ میلی ثانیه افزایش یافته است، زیرا Replica اصلی باید منتظر دریافت تأیید از اکثریت Replica ها بماند. با این حال، هیچ Stale Read مشاهده نشد و تمام عملیات GET جدیدترین مقدار را بازگرداندند.

```
-consistency=strong & -delay=500ms
```

```
PUT Latency: 701.638317ms
```

```
GET Latency (last successful): ~1.658964ms
```

```
Stale Reads Count: 0
```

```
Convergence Time: 1.6589ms
```

این آزمایش نشان می‌دهد که بین Consistency و Performance یک Trade-off وجود دارد. Eventual Consistency زمان پاسخ نوشتن کمتری دارد اما ممکن است منجر به Stale Read شود، در حالی که Strong Consistency با افزایش زمان پاسخ نوشتن، سازگاری بیشتری را تضمین می‌کند.

جدول زیر، مقادیر هر یک از متریک‌ها را در هر کدام از سناریوهای گفته شده نشان می‌دهد:

Scenario	Model	PUT Latency	GET Latency	Convergence Time	Stale Reads
Scenario 1	Eventual	بسیار کم	متغیر	حدود ۳ ثانیه	مشاهده شد
Scenario 2	Eventual	کم	کم	وابسته به رپلیکاهای فعال	ممکن است رخ دهد
Scenario 2	Strong	بیشتر از حالت Eventual	کم	سریع پس از رسیدن به Quorum	مشاهده نشد
Scenario 3	Eventual	کم	کم	پس از تبادل پیام‌ها	وابسته به زمان یکسان شدن
Scenario 4.1	Eventual (500ms)	7.99 ms	حدود 75 میلی ثانیه	602 ms	7
Scenario 4.2	Eventual (2000ms)	1.49 ms	حدود 71 میلی ثانیه	2.05 s	28
Scenario 4.3	Strong (500ms)	701 ms	1.66 ms	1.66 ms	0

مشکلات و محدودیت‌های پیاده‌سازی

در خصوص محدودیت‌های پیاده‌سازی در این تمرین می‌توان به چند مورد اشاره کرد. در این تمرین، برای ساده‌سازی، از پروتکل HTTP و یک مکانیزم ساده مبتنی بر Quorum استفاده شده است و الگوریتم‌های پیشرفته‌ای مانند Paxos یا Raft پیاده‌سازی نشده‌اند. همچنین داده‌ها فقط در حافظه نگهداری می‌شوند و با خاموش شدن Replica از بین می‌روند. علاوه بر این، در مدل Strong Consistency مکانیزم Rollback برای بازگرداندن تغییرات در صورت شکست عملیات وجود ندارد و هدف اصلی این پروژه صرفاً نمایش مفاهیم Replication و Consistency بوده است.